ISLS Extensions

Phase 10 – 12
v1.0.0

Phase 10: Multi-Language Forge
Phase 11: The Alchemist
Phase 12: Seraphic Self-Calibration

From Rust-only to Full-Stack.
From Templates to Combinatorial Discovery.
From Manual Tuning to Self-Optimization.

Sebastian Klemm

18 March 2026

Abstract

Three phases that complete the ISLS Forge as a universal software fabrication platform.

Phase 10 extends the Forge, Foundry, and Template system from Rust-only to multi-language output: TypeScript/React for frontends, Python for scripts and ML, SQL for database schemas, HTML/CSS for static sites, YAML/Docker/CI for operations. New matrices, new toolchain executors, new full-stack templates that produce complete deployable applications.

Phase 11 introduces the Alchemist: an infogenetic signature system where every pattern, template, and crystal carries a rich multi-dimensional signature that can be decomposed, recombined, and mutated like genetic material. A combinatorial exploration engine (the "slot machine") discovers novel architectures by mixing signature components from different domains. Keyspace shaving identifies the most promising regions. Singularity detection flags configurations that score anomalously high — unexplored gold.

Phase 12 absorbs the Seraphic Calibration Module: the Forge tunes itself. Every forge run produces benchmark data. The Seraphic feedback loop uses this data to adjust PMHD strategies, Oracle prompts, template selection, and quality thresholds — converging toward

optimal fabrication performance through a contractive double-kick operator with Proof-of-Resonance gating.

Status: Implementation Specification.
New Crates: C28 (isls-multilang), C29 (isls-alchemist), C30 (isls-calibrate).
Depends on: C23–C27 (Forge stack), C25 (Oracle).

Contents

I Phase 10: Multi-Language Forge 4

1 Purpose 4

2 C28: `isls-multilang` 4

3 Full-Stack Templates 5
3.1 T11: SaaS Starter (Full-Stack) 5

4 Cross-Language Derive 6

5 Acceptance Tests (Phase 10) 6

II Phase 11: The Alchemist 7

6 Purpose 7

7 C29: `isls-alchemist` 7
7.1 Infogenetic Signatures 7
7.2 Recombination Engine 8
7.3 Keyspace Shaving 9
7.4 Singularity Detection 9
7.5 The Alchemy Lab (Studio Integration) 9

8 Acceptance Tests (Phase 11) 10

III Phase 12: Seraphic Self-Calibration 10

9 Purpose 10

10 C30: `isls-calibrate` 11

11 Studio Integration 12

12 Acceptance Tests (Phase 12) 13

IV Dependency Graph and Handoff 13

13 New Crates 13

14 Test Count 14

15 Completion Criteria 14

Part I
Phase 10: Multi-Language Forge

1 Purpose

The Forge currently produces Rust code through the RustModuleMatrix. Real software projects are multi-language: a Rust backend needs a TypeScript frontend, SQL migrations, Docker deployment, CI pipelines. Phase 10 adds language-specific matrices, toolchain executors, and full-stack templates so that a single DecisionSpec produces a complete, multi-language, deployable project.

2 C28: `isls-multilang`

Definition 2.1 (Language Support)**.** Phase 10 adds support for six additional output languages:

Language	Matrix	What It Produces
TypeScript/React	`ReactMatrix`	Components, hooks, pages, stores, types. JSX/TSX with Tailwind CSS.
Python	`PythonMatrix`	Modules, classes, FastAPI endpoints, CLI scripts, dataclasses, pytest tests.
SQL	`SqlMatrix`	DDL schemas, migrations, indexes, views, stored procedures, seed data.
HTML/CSS	`StaticMatrix`	Static pages, templates (Jinja2 or Handlebars), CSS with custom properties.
YAML/Docker	`OpsMatrix`	Dockerfiles, docker-compose, GitHub Actions, Kubernetes manifests, .env templates.
Markdown	`DocsMatrix`	README, API docs, architecture docs, changelogs, contribution guides.

Definition 2.2 (Toolchain Executors)**.** The Foundry (C27) gains new toolchain executors:

```
pub enum Toolchain {
    Rust,            // cargo check, cargo test, cargo clippy
    TypeScript,      // tsc --noEmit, npm test (or vitest)
    Python,          // python -m py_compile, pytest
    Sql,             // sqlx-cli migrate (or dry parse validation)
    Docker,          // docker build --check (syntax)
    Yaml,            // YAML lint (built-in serde_yaml parse)
    Markdown,        // exists check only (always valid)
}

impl ToolchainExecutor {
    pub fn check(&self, dir: &Path, lang: &Toolchain) -> ToolchainResult
        ;
    pub fn test(&self, dir: &Path, lang: &Toolchain) -> ToolchainResult;
    pub fn lint(&self, dir: &Path, lang: &Toolchain) -> ToolchainResult;
    pub fn fmt(&self, dir: &Path, lang: &Toolchain) -> ToolchainResult;
}
```

Toolchains are optional. If `tsc` is not installed, TypeScript atoms are validated by the built-in syntax check (serde parse of the AST structure). If `python` is not installed, Python atoms are validated structurally. The system never *requires* external tools — it degrades gracefully.

3 Full-Stack Templates

Six new templates are added to the catalog (C26):

ID	Name	Atoms	Mol.	Languages
T11	SaaS Starter	22	6	Rust + React + SQL + Docker + CI
T12	Dashboard App	18	5	Rust + React + SQL + Docker
T13	API + Docs	12	3	Rust + Markdown + Docker
T14	Python ML Pipeline	10	3	Python + SQL + Docker
T15	Static Site	8	2	HTML/CSS + Markdown + CI
T16	Monorepo Scaffold	16	5	Rust + TS + SQL + Docker + CI

3.1 T11: SaaS Starter (Full-Stack)

T11 Composition Tree

```
System: saas-starter
  |
  +-- Molecule: backend (Rust)
  |     +-- Atom: api_router      [Rust, Oracle]
  |     +-- Atom: auth_service    [Rust, Oracle]
  |     +-- Atom: domain_models   [Rust, Oracle]
  |     +-- Atom: db_layer        [Rust, Oracle]
  |     +-- Atom: config          [Rust, Static]
  |     +-- Atom: main            [Rust, Static]
  |
  +-- Molecule: frontend (TypeScript/React)
  |     +-- Atom: app_shell       [TSX, Static]    -- layout, router,
     providers
  |     +-- Atom: auth_pages      [TSX, Oracle]    -- login, register,
     profile
  |     +-- Atom: dashboard_page  [TSX, Oracle]    -- main feature page
  |     +-- Atom: api_client      [TS, Derive]     -- generated from
     backend DTOs
  |     +-- Atom: components      [TSX, Oracle]    -- shared UI
     components
  |     +-- Atom: hooks           [TS, Oracle]     -- React hooks, state
      management
  |
  +-- Molecule: database (SQL)
  |     +-- Atom: schema          [SQL, Oracle]    -- CREATE TABLE
     statements
  |     +-- Atom: migrations      [SQL, Oracle]    -- migration files
  |     +-- Atom: seeds           [SQL, Oracle]    -- development seed
     data
  |
  +-- Molecule: ops (Docker/CI)
  |     +-- Atom: dockerfile      [Docker, Static]
  |     +-- Atom: compose         [YAML, Static]
  |     +-- Atom: ci_pipeline     [YAML, Static]  -- GitHub Actions
  |
  +-- Molecule: docs (Markdown)
        +-- Atom: readme          [MD, Derive]     -- from spec
        +-- Atom: api_docs        [MD, Derive]     -- from backend
           routes
```

Interfaces:

- Frontend `api_client` derives from backend DTOs (type-safe client generation).
- Backend `db_layer` consumes SQL `schema`.
- Docker `compose` references backend, frontend, and database services.
- CI `pipeline` runs backend tests + frontend build + DB migrations.

Output: A monorepo directory with `backend/`, `frontend/`, `database/`, `docker/`, `docs/`, `.github/workflows/`.

4 Cross-Language Derive

Definition 4.1 (Cross-Language Derive)**.** A *Derive* fill strategy can cross language boundaries:

```
pub struct CrossDerive {
    pub source_atom: String,      // e.g., "domain_models" (Rust)
    pub target_language: String,  // e.g., "typescript"
    pub transform: DeriveTransform,
}

pub enum DeriveTransform {
    /// Extract public types and generate equivalent types
    TypeMirror,
    /// Extract API routes and generate a client
    ApiClient,
    /// Extract DB schema and generate ORM models
    OrmModels,
    /// Extract types and generate JSON Schema
    JsonSchema,
    /// Extract spec and generate documentation
    DocGen,
}
```

Example: `domain_models` (Rust structs with serde derives) is transformed into TypeScript interfaces automatically. No Oracle call needed — pure structural transformation.

5 Acceptance Tests (Phase 10)

ID	Name	Procedure
AT-ML1	TypeScript output	Forge with ReactMatrix; verify .tsx files produced.
AT-ML2	Python output	Forge with PythonMatrix; verify .py files.
AT-ML3	SQL output	Forge with SqlMatrix; verify .sql files.
AT-ML4	Cross-derive	Forge Rust backend; derive TS client; verify types match.
AT-ML5	Full-stack T11	Forge SaaS Starter; verify backend/, frontend/, database/, docker/, docs/ dirs.

AT-ML6	Multi-toolchain	Run Foundry on T11 output; verify Rust compiles and TS type-checks (or degrades gracefully).
AT-ML7	YAML validation	Forge docker-compose; verify valid YAML.
AT-ML8	Monorepo scaffold	Forge T16; verify workspace Cargo.toml + package.json + docker-compose.

Part II
Phase 11: The Alchemist

6 Purpose

Templates are predefined. Patterns are learned from past runs. But neither discovers *novel* architectures. The Alchemist is the combinatorial exploration engine that treats architectural decisions as a searchable space — mixing, recombining, and mutating structural signatures to find configurations that no one designed on purpose but that score anomalously high on quality metrics.

This is the "slot machine" for structural singularities.

7 C29: `isls-alchemist`

7.1 Infogenetic Signatures

Definition 7.1 (Infogenetic Signature)**.** Every template, pattern, and crystal carries a rich signature:

```
pub struct InfogeneticSignature {
    /// 5D embedding (existing)
    pub embedding: FiveDState,

    /// Structural genome: ordered list of component archetypes
    pub genome: Vec<Gene>,

    /// Interaction matrix: pairwise coupling strengths between genes
    pub interactions: Vec<Vec<f64>>,

    /// Spectral fingerprint: eigenvalues of the interaction matrix
    pub spectrum: Vec<f64>,

    /// Domain tags with weights
    pub phenotype: BTreeMap<String, f64>,

    /// Quality triplet (psi, rho, omega) if measured
    pub triplet: Option<(f64, f64, f64)>,
}

pub struct Gene {
    pub id: String,
```

```
    pub kind: GeneKind,
    pub properties: BTreeMap<String, f64>,
}

pub enum GeneKind {
    Router, Handler, Service, Model, Database, Auth,
    Cache, Queue, WebSocket, Scheduler, Logger, Config,
    Frontend, Component, Hook, Store, Page,
    Migration, Schema, Seed,
    Docker, CI, Deploy,
    Custom(String),
}
```

Remark 7.1. The infogenetic signature is the "DNA" of a software architecture. Two architectures with similar genomes and similar interaction matrices will have similar spectral fingerprints — even if they were built for different domains. This is what enables cross-domain transfer: a pattern that works for an e-commerce backend can be adapted for a healthcare API if their infogenetic signatures are close.

7.2 Recombination Engine

Definition 7.2 (Recombination Operations)**.**

```
pub enum AlchemyOperation {
    /// Cross two signatures: take genes from parent A and parent B
    Crossover {
        parent_a: Hash256,
        parent_b: Hash256,
        crossover_point: usize,
    },

    /// Mutate a single gene (change kind, adjust properties)
    Mutate {
        source: Hash256,
        gene_index: usize,
        mutation: GeneMutation,
    },

    /// Insert a gene from another signature into a host
    Splice {
        host: Hash256,
        donor: Hash256,
        donor_gene: usize,
        insert_at: usize,
    },

    /// Remove a gene and repair interfaces
    Excise {
        source: Hash256,
        gene_index: usize,
    },

    /// Blend: weighted average of two signatures' interaction matrices
    Blend {
        sig_a: Hash256,
        sig_b: Hash256,
```

```
        alpha: f64,  // 0.0 = pure A, 1.0 = pure B
    },
}
```

7.3 Keyspace Shaving

Definition 7.3 (Keyspace)**.** The keyspace K is the set of all possible infogenetic signatures reachable from the current pattern memory through recombination operations. Keyspace shaving is the process of progressively narrowing K to the most promising region:

1. Start with K_0 = all signatures in pattern memory.
2. Select the operation with highest expected quality gain: $q(g) = \frac{H(K)-H(K\cap g)}{\text{risk}(g)+\varepsilon}$
3. Apply the operation: $K_{t+1} = K_t \cap g^*$.
4. Repeat until $\text{Var}(K_t) \leq \tau$ or budget exhausted.

7.4 Singularity Detection

Definition 7.4 (Singularity)**.** A singularity is a discovered signature that scores $> 2\sigma$ above the mean quality of all patterns in its spectral neighborhood:

$$\text{singularity}(\mathbf{s}) = \frac{Q(\mathbf{s}) - \mu_{\text{neighbors}}}{\sigma_{\text{neighbors}}} > 2$$

where Q is the quality functional and the neighborhood is defined by spectral fingerprint distance $< \theta_{\text{spec}}$.

A singularity represents unexplored gold: an architecture that is significantly better than anything the system has seen in that region of signature space.

7.5 The Alchemy Lab (Studio Integration)

New Studio View: The Alchemy Lab

```
+-----------------------------------------------------+
|  ALCHEMY LAB                                         |
+-----------------------------------------------------+
|                                                      |
|  +-- Signature Viewer ----------------------------+ |
|  | Selected: pattern p-a7b3 (REST API)            | |
|  | Genome: [Router, Auth, Service, Model, DB]     | |
|  | Spectrum: [0.91, 0.73, 0.45, 0.22, 0.08]      | |
|  | Quality: psi=0.82 rho=0.76 omega=0.91          | |
|  +-----------------------------------------------+ |
|                                                      |
|  +-- Recombination Panel -------------------------+ |
|  | Parent A: [p-a7b3 (REST API)     v]            | |
|  | Parent B: [p-f2d1 (WebSocket)    v]            | |
|  | Operation: [Crossover v] at gene 3             | |
|  |                                                 | |
|  | [ RECOMBINE ] -> Preview child signature        | |
|  | [ FORGE CHILD ] -> Materialize as real project  | |
|  +-----------------------------------------------+ |
|                                                      |
|  +-- Keyspace Map (canvas) ----------------------+ |
|  | [2D projection of signature space]              | |
```

```
|   | Dots = patterns, Stars = singularities          | |
|   | Color = quality score                           | |
|   +-------------------------------------------------+ |
|                                                       |
|   +-- Singularity Feed ----------------------------+ |
|   | ! SINGULARITY: REST+WebSocket+Queue hybrid      | |
|   |   Score: 0.94 (2.3 sigma above neighborhood)   | |
|   |   [Inspect] [Forge] [Save as Template]          | |
|   +-------------------------------------------------+ |
+-----------------------------------------------------+
```

8 Acceptance Tests (Phase 11)

ID	Name	Procedure
AT-AL1	Signature creation	Create InfogeneticSignature from a template; verify genome and spectrum.
AT-AL2	Crossover	Cross two signatures; verify child has genes from both parents.
AT-AL3	Mutation	Mutate a gene; verify signature changes.
AT-AL4	Splice	Insert a gene from donor; verify interfaces repaired.
AT-AL5	Keyspace shaving	Shave from 100 signatures; verify variance decreases.
AT-AL6	Singularity detection	Inject a high-scoring outlier; verify detected as singularity.
AT-AL7	Forge from recombination	Recombine two signatures; forge the child; verify working project.
AT-AL8	Spectral fingerprint	Verify two similar templates have similar spectral fingerprints.

Part III
Phase 12: Seraphic Self-Calibration

9 Purpose

The Forge has many knobs: PMHD tick count, opposition strength, quality thresholds, Oracle temperature, template match threshold, retry budget, fill strategy weights. Currently these are hand-configured. Phase 12 makes the Forge tune itself.

The Seraphic Calibration Shell (absorbed from the Q⊗DASH SCS document) wraps the Forge in a feedback loop: every forge run produces benchmark data (compilation success rate, test pass rate, Oracle rejection rate, token usage, autonomy ratio). This data feeds a contractive double-kick operator that adjusts the Forge's configuration toward a fixpoint attractor — the optimal operating point for the current domain and workload.

10 C30: isls-calibrate

Definition 10.1 (Forge Configuration Space)**.** The configuration space C includes:

```
pub struct ForgeCalibrationState {
    // PMHD
    pub pmhd_ticks: u64,
    pub pmhd_pool_size: usize,
    pub pmhd_opposition_strength: f64,
    pub pmhd_mutation_rate: f64,
    pub pmhd_strategy: SeedStrategy,

    // Oracle
    pub oracle_temperature: f64,
    pub oracle_max_tokens: usize,
    pub oracle_max_retries: usize,

    // Templates
    pub template_match_threshold: f64,

    // Quality
    pub quality_thresholds: QualityMetrics,

    // Foundry
    pub foundry_max_attempts: usize,
}
```

Definition 10.2 (Performance Triplet)**.** For each forge run, a performance triplet is computed:

```
pub struct ForgeTriplet {
    pub psi: f64,   // quality: test pass rate * constraint satisfaction
    pub rho: f64,   // stability: 1 - (variance across retries)
    pub omega: f64,  // efficiency: 1 / (tokens_used / baseline)
}
```

Definition 10.3 (Seraphic Feedback)**.** After each forge run, the benchmark vector u is encoded into the calibration field:

```
pub struct CalibrationField {
    pub state: Vec<f64>,       // M(t) field state
    pub history: Vec<ForgeTriplet>, // past triplets
}

impl CalibrationField {
    pub fn update(&mut self, triplet: &ForgeTriplet, injection: &[f64])
       {
        // M(t+1) = Norm(alpha * M(t) + gamma * I_t)
        for i in 0..self.state.len() {
            self.state[i] = (0.9 * self.state[i]
                            + 0.1 * injection[i]).clamp(0.0, 1.0);
        }
        self.history.push(triplet.clone());
    }
}
```

Definition 10.4 (Double-Kick Operator)**.** The configuration update operator $T = \Phi_V \circ \Phi_U$:

```
pub fn double_kick(
    config: &mut ForgeCalibrationState,
    triplet: &ForgeTriplet,
    field: &CalibrationField,
) {
    // Phi_U: quality-ascending update
    if triplet.psi < target_psi {
        config.pmhd_ticks += 100;          // more drill
        config.oracle_max_retries += 1;     // more attempts
    }
    if triplet.psi > target_psi * 1.1 {
        config.pmhd_ticks = (config.pmhd_ticks - 50).max(100);
        config.oracle_max_retries = (config.oracle_max_retries - 1).max
            (1);
    }

    // Phi_V: stability/efficiency stabilization
    if triplet.rho < target_rho {
        config.pmhd_opposition_strength += 0.05;
        config.oracle_temperature = (config.oracle_temperature - 0.05).
            max(0.0);
    }
    if triplet.omega < target_omega {
        config.oracle_max_tokens = (config.oracle_max_tokens - 256).max
            (1024);
        config.template_match_threshold -= 0.05; // use templates more
    }

    // Clamp all values to admissible ranges
    config.clamp();
}
```

Definition 10.5 (Proof-of-Resonance for Config)**.** A config update is accepted only if:

1. $\psi(c') \geq \psi(c)$ (no quality regression),
2. $\rho(c')$ within tolerance of $\rho(c)$,
3. $\omega(c')$ above minimum threshold,
4. Field $M(t)$ shows positive correlation with the new config's injection.

If PoR fails, the config update is rejected and the system continues with the previous configuration.

Definition 10.6 (CRI Regime Change)**.** If the global quality $J(t) = \psi \cdot \rho \cdot \omega$ stagnates for N consecutive runs, a CRI impulse triggers:

- Switch PMHD strategy (e.g., Hybrid $\rightarrow$ Evolutionary).
- Increase Oracle temperature temporarily (explore more).
- Reset template match threshold to default.
- Log the regime change as a calibration event.

After the CRI impulse, the double-kick resumes in the new region.

11 Studio Integration

Enhanced Dashboard: Calibration Panel

```
+-- Forge Calibration ----------------------------------+
| Triplet: psi=0.87 rho=0.91 omega=0.76                 |
| Global J: 0.60  (trend: +0.03 over last 10 runs)      |
| Regime: Hybrid | Calibration step: 47                  |
| Last CRI: step 23 (Greedy -> Hybrid)                   |
|                                                         |
| [Triplet chart over time: psi/rho/omega lines]         |
| [Config evolution: ticks, temperature, threshold]      |
+-------------------------------------------------------+
```

12 Acceptance Tests (Phase 12)

ID	Name	Procedure
AT-SC1	Triplet computation	Run forge; verify triplet $(\psi, \rho, \omega) \in [0, 1]^3$.
AT-SC2	Field update	Run 5 forges; verify field state changes monotonically toward better configs.
AT-SC3	Double-kick	Start with suboptimal config; verify config adjusts after 3 runs.
AT-SC4	PoR rejection	Mock a config that regresses quality; verify rejected and old config restored.
AT-SC5	CRI trigger	Stagnate J for 10 runs; verify CRI impulse triggers strategy switch.
AT-SC6	Config persistence	Calibrate; restart; verify calibrated config loaded from store.

Part IV
Dependency Graph and Handoff

13 New Crates

Crate	ID	Depends On
`isls-multilang`	C28	C22 (ArtifactIR), C23 (Forge), C26 (Templates), C27 (Foundry)
`isls-alchemist`	C29	C22, C23, C25 (Oracle), C26 (Templates)
`isls-calibrate`	C30	C23 (Forge), C25 (Oracle), C17 (Store)

14 Test Count

Phase	New Tests	Cumulative
Phase 1–9	326	326
Phase 10 (C28)	8	334
Phase 11 (C29)	8	342
Phase 12 (C30)	6	$\geq$ 348

15 Completion Criteria

1. Forge produces TypeScript, Python, SQL, HTML, YAML, Docker, Markdown output.
2. Full-stack templates (T11–T16) produce multi-language projects.
3. Cross-language derive generates TS types from Rust structs.
4. Foundry validates with language-appropriate toolchains.
5. Infogenetic signatures computed for all patterns and templates.
6. Recombination produces viable child signatures.
7. Singularity detection identifies outlier architectures.
8. Alchemy Lab view in Studio with keyspace visualization.
9. Seraphic calibration adjusts forge config across runs.
10. PoR gates config changes; CRI triggers regime switches.
11. Total tests $\geq$ 348, zero warnings.
12. **30 Crates. Every language. Self-tuning. Architecture DNA.**

Generated for ISLS Extensions Phase 10–12 v1.0.0.

Phase 10: The Forge speaks every language.
Phase 11: The Alchemist discovers what no one designed.
Phase 12: The system tunes itself.

Infogenetic signatures. Combinatorial exploration. Fixpoint convergence.
Every pattern has DNA. Every recombination is an experiment.
Every singularity is unexplored gold.